

Supplementary Materials of AnoMon

1 Mathematical Analysis

1.1 Analysis of AnoSketch

We provide the details of our mathematical analysis on AnoSketch. We first prove that AnoSketch achieves unbiased estimation for arbitrary flow group in Appendix 1.1.1. Then we prove that AnoSketch has smaller estimation variance than CocoSketch [1] in Appendix 1.1.2.

1.1.1 Proof of Unbiased Estimation

We will prove that AnoSketch provides unbiased estimation for any flow in Theorem 4.1, and thus the estimation for arbitrary flow group is also unbiased. Given an arbitrary flow f_i . Denote the real frequency of f_i as $F(f_i)$, and denote the estimated frequency of f_i as $\hat{F}(f_i)$. The estimated frequency consists of two parts: 1) the estimated frequency recorded in HotFilter (denoted as $\hat{F}_H(f_i)$), and 2) the estimated frequency recorded in ColdSampler (denoted as $\hat{F}_C(f_i)$). We have $\hat{F}(f_i) = \hat{F}_H(f_i) + \hat{F}_C(f_i)$. We suppose up to now, f_i has been inserted into ColdSampler (or evicted from HotFilter) t times with frequency $w_1, w_2, \dots, w_t$. Let $F_C(f_i) = \sum_{i=1}^t w_i$.

Lemma 4.1. $F(f_i) = \hat{F}_H(f_i) + F_C(f_i)$ holds for any flow f_i .

Proof. We prove this lemma by induction on the number of inserted packets k .

Base case: When AnoSketch is empty, the lemma holds obviously.

Induction step: Suppose the lemma holds for the first $k - 1$ insertions. Now we consider the k_{th} insertion. Suppose that the incoming packet is of flow f_k and it is mapped to a bucket in HotFilter recording flow ID f_j and positive votes p_j . We discuss the following three cases:

Case 1: $f_k = f_j$. In this case, only one flow f_j is affected by this insertion, so we just need to prove that the lemma holds for f_j , namely $F(f_j) = \hat{F}_H(f_j) + F_C(f_j)$. In this case, p_j is increased by one, namely $\hat{F}_H(f_j)$ is increased by one, and we do not insert any item to ColdSampler, so $F_C(f_j)$ is unchanged. Therefore, the increment on RHS is one, which is equal to the increment on LHS.

Case 2: $f_k \neq f_j$ and no replacement occurs in HotFilter. We need to prove that both $F(f_j) = \hat{F}_H(f_j) + F_C(f_j)$ and $F(f_k) = \hat{F}_H(f_k) + F_C(f_k)$ still hold after insertion. As there is no replacement in HotFilter, we do not modify p_j and just insert $(f_k, w = 1)$ to ColdSampler. For f_j , there is no change on either side of the equation, so the lemma still holds. For f_k , notice that $F_C(f_k)$ is increased by one, and $\hat{F}_H(f_k)$ remains zero. Therefore, the increment of RHS is one, which is equal to the increment of LHS. The lemma holds for f_k .

Case 3: $f_k \neq f_j$ and f_j is replaced by f_k in HotFilter. We need to prove that both $F(f_j) = \hat{F}_H(f_j) + F_C(f_j)$ and $F(f_k) = \hat{F}_H(f_k) + F_C(f_k)$ still hold after insertion. In this case, the hashed bucket in HotFilter is changed from $\langle f_j, p_j \rangle$ to $\langle f_k, 1 \rangle$, and we insert (f_j, p_j) to ColdSampler. We first consider the former equation. We can see that $F(f_j)$ is unchanged, $\hat{F}_H(f_j)$ is subtracted by p_j , and $F_C(f_j)$ is increased by p_j . Thus, both LHS and RHS are unchanged, so the equation still holds. For the latter equation, we can see that $F(f_k)$ and $\hat{F}_H(f_k)$ are both increased by one, and $F_C(f_k)$ is unchanged. Thus, both LHS and RHS are increased by one, so the equation still holds.

Conclusion: We have proved that both the base case and the induction step hold. Therefore, the lemma holds for any number of inserted packets k . □

Lemma 4.2. $\mathbb{E} [\hat{F}_C(f_i)] = F_C(f_i)$ for any flow f_i .

Proof. We prove this lemma by induction on the number of insertions to ColdSampler k .

Base case: when the ColdSampler is empty, the lemma holds obviously.

Induction step: Suppose the lemma holds for the first $k - 1$ insertions to ColdSampler. Now we consider the k_{th} insertion. Suppose that the incoming item is (f_k, w) and it is mapped to a bucket recording flow ID f_j and frequency $\mathcal{F}_j$. We update the mapped bucket $\mathcal{B}[h_2(f_k)]$ to $\langle f', F' \rangle$ as below.

$$\langle f', F' \rangle = \begin{cases} \langle f_k, \mathcal{F}_j + w \rangle, & \text{w.p. } \frac{w}{\mathcal{F}_j + w} \\ \langle f_j, \mathcal{F}_j + w \rangle, & \text{w.p. } \frac{\mathcal{F}_j}{\mathcal{F}_j + w} \end{cases} \quad (1)$$

After update, $\hat{F}_C(f_k)$ has a probability of $\frac{w}{\mathcal{F}_j + w}$ to change from 0 to $\mathcal{F}_j + w$, and has a probability of $\frac{\mathcal{F}_j}{\mathcal{F}_j + w}$ to be unchanged. Thus, $\mathbb{E}(\hat{F}_C(f_k))$ is increased by $\frac{w}{\mathcal{F}_j + w} \times (\mathcal{F}_j + w) = w$, which is equal to the increment of $F_C(f_k)$. On the other hand, $\hat{F}_C(f_j)$ has a probability of $\frac{w}{\mathcal{F}_j + w}$ to change from $\mathcal{F}_j$ to 0, and has a probability of $\frac{\mathcal{F}_j}{\mathcal{F}_j + w}$ to be increased by w . Thus, $\mathbb{E}(\hat{F}_C(f_j))$ is increased by $\frac{w}{\mathcal{F}_j + w} \cdot (-\mathcal{F}_j) + \frac{\mathcal{F}_j}{\mathcal{F}_j + w} \cdot w = 0$, and we note that $F_C(f_j)$ is also unchanged. In summary, the lemma still holds after the k_{th} insertion.

Conclusion: We have proved that both the base case and the induction step hold. Therefore, the lemma holds for any number of insertions k . $\square$

Theorem 4.1. For any flow f_i , its estimation frequency $\hat{F}(f_i)$ is unbiased, i.e., $\mathbb{E}[\hat{F}(f_i)] = F(f_i)$.

Proof. According to the query workflow of AnoSketch, we have $\hat{F}(f_i) = \hat{F}_H(f_i) + \hat{F}_C(f_i)$, where $\hat{F}_H(f_i)$ is a fixed value, and $\hat{F}_C(f_i)$ is a random variable. From Lemma 4.1 and Lemma 4.2, we have that

$$\begin{aligned} \mathbb{E}[\hat{F}(f_i)] &= \mathbb{E}[\hat{F}_H(f_i) + \hat{F}_C(f_i)] = \hat{F}_H(f_i) + \mathbb{E}[\hat{F}_C(f_i)] \\ &= \hat{F}_H(f_i) + F_C(f_i) = F(f_i) \end{aligned} \quad (2)$$

$\square$

1.1.2 Proof of Smaller Variance

We prove that AnoSketch has smaller variance for estimating arbitrary flow group than state-of-the-art CocoSketch [1]. In our analysis, we cast the problem of measuring flow group to the problem of subset sum estimation.

Variance for subset sum estimation: Let $F(f)$ be the real frequency of flow f , and $\hat{F}(f)$ be the estimated frequency of f . The estimation variance for subset sum estimation is defined as $Var := \sum_f (F(f) - \hat{F}(f))^2$. We consider the variance increment of inserting each item (f_i, w) to ColdSampler/CocoSketch: $\Delta Var := \sum_f \Delta (F(f) - \hat{F}(f))^2$.

Lemma 4.3. Suppose the mapped bucket is $\langle f_j, \mathcal{F}_j \rangle$, the increment of variance sum caused by inserting (f_i, w) is

$$\Delta Var = \sum_f \Delta (F(f) - \hat{F}(f))^2 = \begin{cases} 2w\mathcal{F}_j, & f_i \neq f_j \\ 0, & f_i = f_j \end{cases} \quad (3)$$

Proof. If $f_i = f_j$, we can directly update the mapped bucket to $\langle f_j, \mathcal{F}_j + w \rangle$, and the variance increment is 0.

If $f_i \neq f_j$, let $p = \frac{w}{\mathcal{F}_j + w}$, which is also the probability of replacing f_j with f_i in Eq.(3), and we have

$$\begin{aligned} \Delta Var &= p \left(\left(\frac{w}{p} - w \right)^2 + \mathcal{F}_j^2 \right) + (1 - p) \left(w^2 + \left(\frac{\mathcal{F}_j}{1 - p} - \mathcal{F}_j \right)^2 \right) \\ &= \frac{w^2}{p} - w^2 + \frac{\mathcal{F}_j^2}{1 - p} - \mathcal{F}_j^2 = 2w\mathcal{F}_j \end{aligned} \quad (4)$$

In this way, we prove that Eq.(3) always holds. $\square$

Theorem 4.2. For any flow f_i , AnoSketch has smaller estimation variance $Var[\hat{F}(f_i)]$ than CocoSketch [1].

Proof. From Lemma 4.3, we can see that the increment of variance sum caused by a single insertion is only related to w and $\mathcal{F}_j$. The smaller the frequency in the mapped bucket (namely $\mathcal{F}_j$), the smaller the variance increment will be. As AnoSketch uses HotFilter to filter large flows, the frequencies in the buckets of ColdSampler will become smaller. Thus, compared to basic CocoSketch, the variance increment of AnoSketch will also become smaller. $\square$

1.2 Analysis of CellSketch

We derive the error upper bound of CellSketch in Theorem 5.1. As multiple flows can collide into one histogram in CellSketch, which makes it hard to derive the accurate error bound. To simply the problem, we neglect the error incurred by hash collisions. We note that in practice, the distribution of most attributes (queuing delay, packet interval, *etc.*) is highly skewed. Take queuing delay as an example. Only a small fraction of packets have large queuing delay, and most packets have small delay. Therefore, the first (leftmost) counter in each histogram of CellSketch has the highest chance to overflow. In the following, we assume that only the leftmost counter can overflow to derive an error upper bound. Note that without this assumption, the error bound in Theorem 5.1 also holds, but the gap between this bound and the real error will become larger. We ignore the error introduced by counter overflow, which is small and can be tuned by changing the overflow threshold.

Theorem 5.1. *Consider a histogram $\mathcal{C}_l[h]$ in CellSketch with r counters whose interval lengths follow a geometric sequence with a ratio of α . Let x be the real p -quantile ($0 < p < 1$), and suppose x falls into $\mathcal{C}_l[h][k]$. We have the relative error $\mathcal{E} := \frac{|x - \hat{x}|}{x}$ is bounded by $\mathcal{E} \leq \frac{\alpha^k - \alpha^{k-1}}{\alpha^k + \alpha^{k-1} - 2}$.*

Proof. We consider the k_{th} counter $\mathcal{C}_l[h][k]$. We suppose the interval length of $\mathcal{C}_l[h][k]$ is $\alpha^{k-1} \cdot T$. Suppose the first counter's interval starts from 0. We can calculate the left endpoint of $\mathcal{C}_l[h][k]$'s interval as $L = \sum_{i=1}^{k-1} \alpha^{i-1} \cdot T = \frac{1 - \alpha^{k-1}}{1 - \alpha} \cdot T$, and the right endpoint of its interval as $R = \sum_{i=1}^k \alpha^{i-1} \cdot T = \frac{1 - \alpha^k}{1 - \alpha} \cdot T$.

To query the p -quantile, CellSketch reports an estimated value as $\hat{x} = \frac{2}{\frac{1}{L} + \frac{1}{R}} = \frac{2LR}{L+R}$. We calculate the reative error under the following two cases.

Case 1: $x \leq \hat{x}$:

$$\mathcal{E} = \frac{\hat{x} - x}{x} = \frac{\hat{x}}{x} - 1 \leq \frac{\hat{x}}{L} - 1 = \frac{R - L}{L + R} = \frac{\alpha^k - \alpha^{k-1}}{\alpha^k + \alpha^{k-1} - 2} \quad (5)$$

Case 2: $x > \hat{x}$:

$$\mathcal{E} = \frac{x - \hat{x}}{x} = 1 - \frac{\hat{x}}{x} \leq 1 - \frac{\hat{x}}{R} = \frac{R - L}{L + R} = \frac{\alpha^k - \alpha^{k-1}}{\alpha^k + \alpha^{k-1} - 2} \quad (6)$$

We can see that in both the two cases above, we can derive the same conclusion that $\mathcal{E} \leq \frac{\alpha^k - \alpha^{k-1}}{\alpha^k + \alpha^{k-1} - 2}$ $\square$

Discussion: Theorem 5.1 gives an error upper bound of one histogram in CellSketch. We can repeatedly use this theorem in the child histograms to derive finer-grained error bounds of CellSketch. From this theorem, we can see that smaller α (ratio of the geometric series) goes with smaller relative error. However, using small α means that we must use more counters to divide the initial interval, which leads to more memory usage. In other words, there exists a trade-off between α and memory usage. In our experiments, we recommend to set $\alpha = 2$, which suffices for good performance.

2 Additional Experimental Results

We present more examples of AnoTable on estimating per-flow *packet interval* distribution in Figure 1, where we select some flows (the top- k largest flows with different k) and plot their real and estimated CDF. We conduct experiments using CAIDA dataset, and we fix the total memory of AnoTable (including CellSketch) to 100KB. We can see that for most flows (especially large flows), their estimated CDF are highly consistent with their real CDF. As AnoTable automatically allocates more memory to large flows and less memory to small flows, the estimated results for small flows might not always be very accurate. Nonetheless, we can see that even for small flows, their estimated CDF also closely match the real CDF.

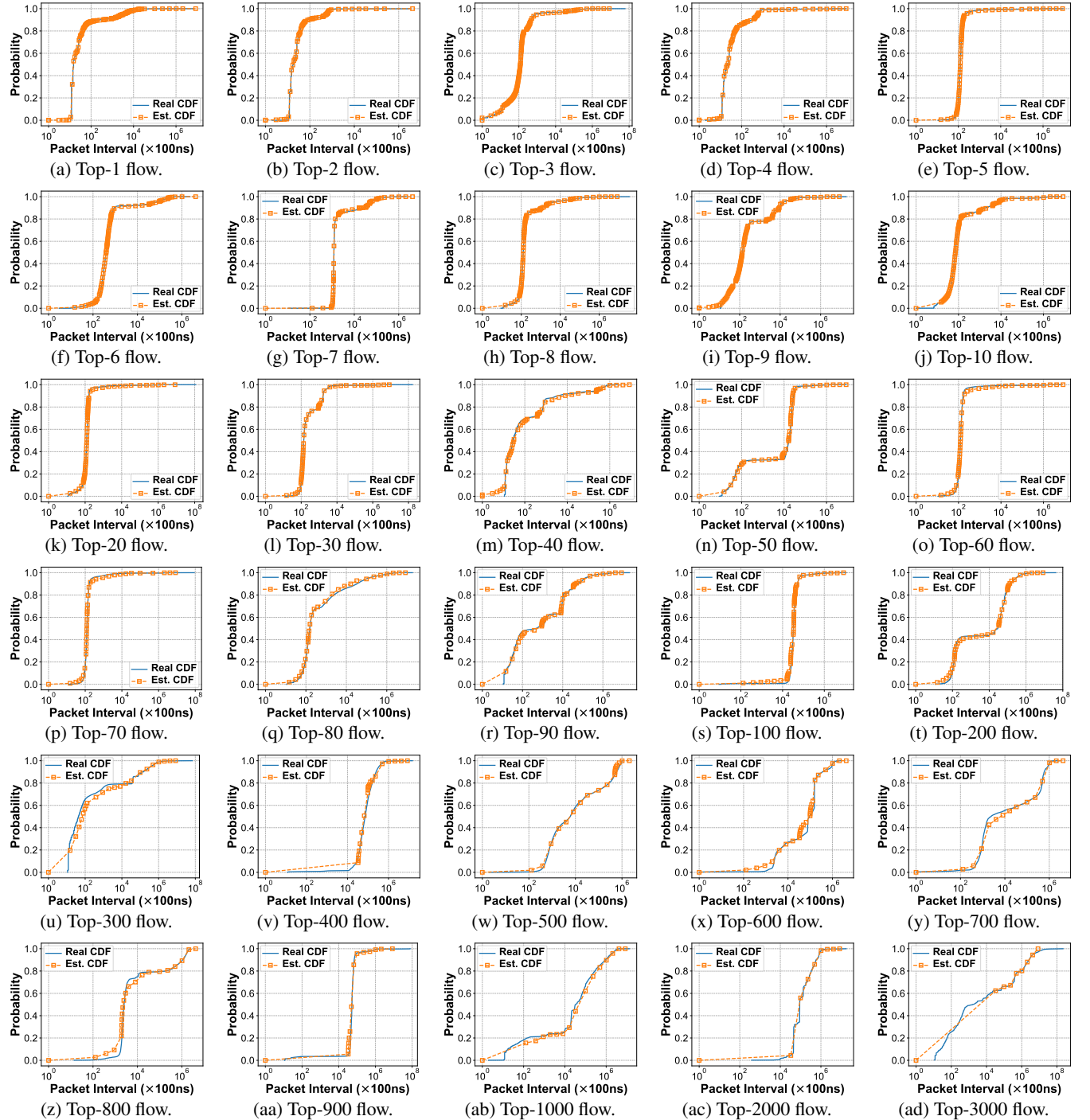


Figure 1: Real and estimated CDF (reported by AnoTable) of different flows.

3 Implementation Details

This section details the data-plane and control-plane implementation of **AnoMon** summarized in § VII of the main paper. We implement **AnoMon** in a testbed of 10 Edgecore Wedge 100BF-32X switches (with Tofino ASIC) and 8 end-hosts in a FatTree topology, with **AnoSketch** and **AnoTable** (with **CellSketch**) on the P4 data plane and the analyzer in a unified DPDK framework.

3.1 Edge Switch PDP Components

We implement **AnoSketch** and the packet-marking function on the programmable data plane at the edge switch; the workflow is shown in Figure 2. We build two copies of each **AnoSketch** (an active sketch and an idle sketch) and use a 0/1 register in Stage 0 to toggle between them based on the epoch-bit. To identify AFG, we build two **AnoSketches** ($AnoSketch_{in}/AnoSketch_{out}$) on each edge switch, recording the packets entering/exiting the network respectively. We implement them by splitting the index of each bucket array into two parts: for an array with $2b$ buckets, the first b buckets serve $AnoSketch_{in}$ and the last b serve $AnoSketch_{out}$, accessed by adding an offset of 0 or b to the hashed index. As shown in Figure 2, we implement **AnoSketch** with a 4-layer **HotFilter** and a 1-layer **ColdSampler**, using 11 ingress and 7 egress stages. Stages 0–1 perform preprocessing (computing hashes, marking packets, and selecting $AnoSketch_{in}/AnoSketch_{out}$); ingress Stages 2–11 implement a 3-layer **HotFilter**; egress Stages 1–3 implement a 4-layer **HotFilter** and Stages 5–8 a 1-layer **ColdSampler**. The **HotFilter** is implemented by modifying the open-sourced **ElasticSketch** [2], whose stage usage we reduce from 4 to 3; each layer holds the *frequencies*, the *votes*/IDs of residing flows, and the drop/pass actions, and replaces the residing flow with the incoming one once the *vote-to-frequency* ratio exceeds λ . Table 1 reports the hardware resource usage on the edge Tofino switch: the average usage is $<14\%$ across all resources, except for Stateful ALUs (used for register access and transactional read-test-write operations).

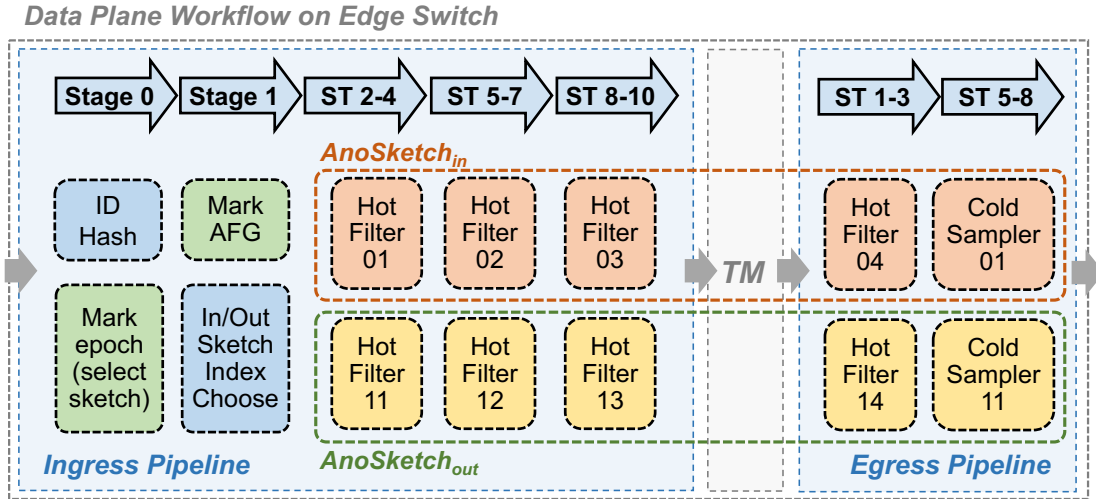


Figure 2: PDP workflow on the edge switch.

Table 1: H/W resource usage on programmable switches.

Resource	Edge switch		Inside-network switch	
	Usage	Percentage	Usage	Percentage
Hash Bits	450	9.83%	212	4.25%
SRAM	51	5.80%	29	3.02%
Map RAM	46	8.71%	26	4.51%
TCAM	0	0%	4	1.39%
Stateful ALU	23	52.27%	13	27.08%
VLIW instr	29	8.24%	20	5.21%
Match Xbar	188	13.35%	87	5.66%

3.2 Inside-network Switch PDP Components

We implement **AnoTable** (with **CellSketch**) on the programmable data plane at the inside-network switch, adding one histogram per bucket to record the per-flow per-hop queuing-delay distribution ($d = 2, l = 1, r = 4, v = 2$); the workflow is shown in Figure 3. One ingress stage implements the forwarding rules. Since the queuing delay of a packet is needed before it is inserted, **AnoTable** is implemented entirely in the egress pipeline: Stage 0 checks the AFG-bit (to decide whether to insert the packet) and toggles between the two sketch copies via a 0/1 register based on the epoch-bit, as in § 3.1; Stages 2–8 implement the $d = 2$ bucket arrays; and when

a histogram counter overflows, it is divided into $v = 2$ histograms in CellSketch (Stages 6–9). The average resource usage on the inside-network switch is $<6\%$ across all resources except Stateful ALUs (Table 1).

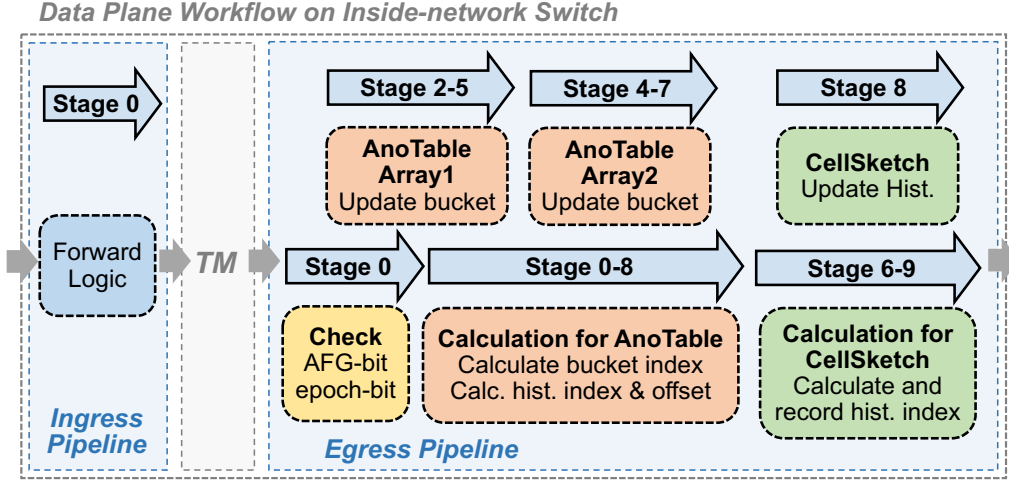


Figure 3: PDP workflow on the inside-network switch.

3.3 Control Plane and Analyzer Components

On the central analyzer, we implement three modules in a unified DPDK framework: 1) a *table collection* module that collects $AnoSketch_{in}$, $AnoSketch_{out}$, and $AnoTable$ from all switches; 2) a *table analysis* module that identifies AFG (§ VI-B of the main paper) and performs the measurement tasks (§ VI-C of the main paper), and also generates notification packets to reconfigure edge switches; and 3) a *notification-packet sending* module.

Collecting tables from the data plane. Using the C++ control-plane API of the Tofino SDK directly takes about 521 ms to collect a 224 KB table, which would severely constrain the epoch length. We instead exploit data-plane features (SALUs, mirror, and recirculate ports): the control plane sends a few tailored packets that are recirculated to access the counters/buckets of each table in turn; every time such a packet accesses a counter/bucket, it uses the SALU to read the value and embed it into its payload, and whenever it reaches its MTU (*e.g.*, 1514 B) the data plane forwards it to the analyzer while the control plane mirrors another packet to read the remaining entries. This reduces the collection time of a 224 KB table to under 2 ms.

Sending tables to the analyzer. After reading the tables, the switch control plane sends them to the analyzer over persistent TCP connections for low-latency, reliable transfer. To collect faster, we separately collect $AnoSketch_{in}$, $AnoSketch_{out}$, and $AnoTable$, refining the procedure of § VI-A of the main paper: 1) when the epoch-bit flips, the analyzer waits a short while (1 ms) to absorb clock-synchronization error and ensure all edge switches have entered the next epoch; 2) it collects all $AnoSketch_{in}$ from the edge switches; 3) it waits another while (9 ms) until all previous-epoch packets have exited the network; 4) it collects all $AnoSketch_{out}$ and $AnoTable$ from the edge and in-network switches. In typical data-center topologies (*e.g.*, leaf-spine, fat-tree), the $AnoTables$ can be collected layer by layer: after collecting $AnoSketch_{in}$, wait until the previous-epoch packets have left the spine switches and collect their $AnoTables$, then wait until they have left the leaf switches and collect theirs.

3.4 Performing the Measurement Tasks

As summarized in § VI-C of the main paper, the analyzer first constructs a query set $\mathcal{F}_q$ of the abnormal flows in AFG (the sampled global heavy changes, operator-reported flows with end-to-end incidents such as TCP duplicate ACKs or timeouts, and selected small business flows of interest), then queries each flow $f_i \in \mathcal{F}_q$ in the $AnoTables$ on the switches along its forwarding path to locate the culprit device or link. **Locating packet drops.** For a victim flow, a slight per-hop packet-count decrease indicates random drops, a sudden drop to 0 indicates a blackhole, and a sudden increase indicates a forwarding loop. **Locating inflated queuing delay.** The analyzer queries the per-hop delay distribution of the victim flow and flags switches with inflated delay, *e.g.*, those on which the 0.8-quantile exceeds 100 μs . **Locating packet-interval jitters.** The analyzer queries the per-hop packet-interval distribution and reports switches where it changes rapidly, *e.g.*, with a suddenly increased 0.9-quantile. **Monitoring user-defined distributions.** By querying the $AnoTables$ (with CellSketch), the analyzer can obtain the distribution of any user-defined attribute at per-flow per-hop granularity, extending $AnoMon$ to network diagnosis, troubleshooting, *etc.*

4 Evaluation Metrics

We define the metrics used in § VIII of the main paper.

1) *Recall Rate (RR)*: the ratio of the number of correctly reported instances to the number of all correct instances.

2) *Precision Rate (PR)*: the ratio of the number of correctly reported instances to the number of all reported instances.

3) F_1 Score: $\frac{2 \times PR \times RR}{PR + RR}$.

4) *Average Relative Error (ARE)* [2]: used to evaluate flow-group size estimation, defined as $ARE := \frac{1}{|\Psi|} \sum_{f_i \in \Psi} \frac{|n_i - \hat{n}_i|}{n_i}$, where n_i and $\hat{n}_i$ are the real and estimated sizes of flow group f_i , and Ψ is the query set of all flow groups.

5) *Average Logarithm Error (ALE)* [3]: used to evaluate distribution estimation and adapted to the skewed distribution of most real-world attributes. For a given flow f_i , the logarithm error on its p -quantile Q_p is $|\log_2 Q_p - \log_2 \hat{Q}_p| = |\log_2 \frac{Q_p}{\hat{Q}_p}|$; the ALE averages this over all flows at their 0.95-quantiles.

5 Sensitivity to Anomaly Intensity

Figure 4 complements the network-anomaly-localization figure (Figure 10) of the main paper, reporting the anomaly-localization F_1 score of AnoMon, LightGuardian, and Marple as the anomaly rate (packet-drop, loop, jitter, or congestion rate) varies, at a fixed per-switch memory. The F_1 score of AnoMon degrades only slightly as the anomaly rate drops, whereas those of the always-on baselines degrade sharply; for example, under a 0.05% packet-drop rate, AnoMon still exceeds 0.8 F_1 while LightGuardian and Marple fall to 0.52 and 0.48.

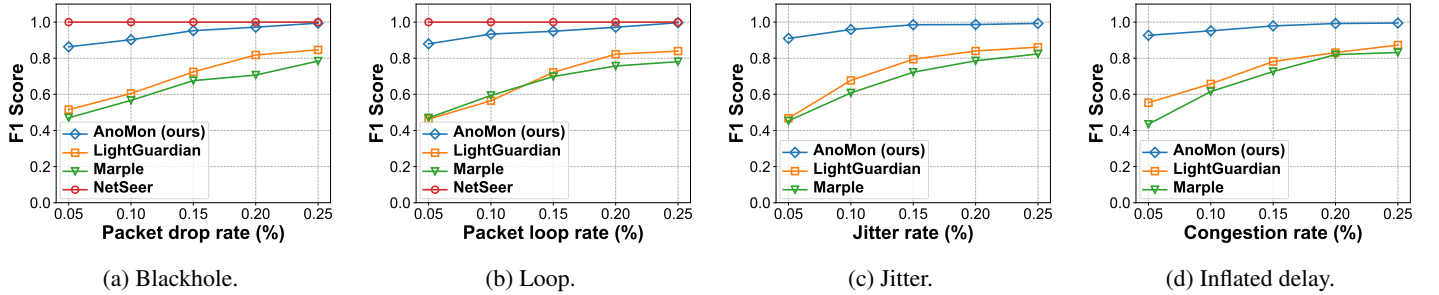


Figure 4: Anomaly-localization F_1 score vs. anomaly intensity, at a fixed per-switch memory (complements the network-anomaly-localization figure (Figure 10) of the main paper).

6 Algorithm Pseudocode

For completeness, this section provides the detailed pseudocode of the insertion and query procedures of AnoSketch, AnoTable, and CellSketch, which are described in the main paper of AnoMon.

Algorithm 1: Insertion workflow of AnoSketch

Input: A packet of flow f_i

```

1 Procedure AnoSketch_Insert( $f_i$ ):
2   | HotFilter_Insert( $f_i$ );
3 Function HotFilter_Insert( $f_i$ ):
4   | if  $\mathcal{H}[h_1(f_i)]$  is empty or  $\mathcal{H}[h_1(f_i)].ID = f_i$  then
5   |   |  $\mathcal{H}[h_1(f_i)].ID \leftarrow f_i$ ;
6   |   |  $\mathcal{H}[h_1(f_i)].freq \leftarrow 1$ ;
7   |   | return;
8   | else
9   |   | //  $\mathcal{H}[h_1(f_i)].ID \neq f_i$ 
10  |   |  $\mathcal{H}[h_1(f_i)].vote \leftarrow 1$ ;
11  |   | if  $\frac{\mathcal{H}[h_1(f_i)].vote}{\mathcal{H}[h_1(f_i)].freq} > \lambda$  then
12  |   |   | ColdSampler_Insert( $\mathcal{H}[h_1(f_i)].ID, \mathcal{H}[h_1(f_i)].freq$ );
13  |   |   |  $\mathcal{H}[h_1(f_i)] \leftarrow \langle f_i, 1, 0 \rangle$ ;
14  |   | else
15  |   |   | ColdSampler_Insert( $f_i, 1$ );
16 Function ColdSampler_Insert( $f_i, w$ ):
17 |  $\mathcal{B}[h_2(f_i)].freq \leftarrow w$ ;
18 | if  $\mathcal{B}[h_2(f_i)]$  is empty then
19 |   |  $\mathcal{B}[h_2(f_i)].ID \leftarrow f_i$ ;
20 | else if  $\mathcal{B}[h_2(f_i)].ID \neq f_i$  then
21 |   |  $\mathcal{B}[h_2(f_i)].ID \leftarrow f_i$  with probability  $p = \frac{w}{\mathcal{B}[h_2(f_i)].freq}$ ;
```

Algorithm 2: Insertion workflow of AnoTable

Input: A packet p_i of flow f_i

```

1 Procedure AnoTable_Insert( $p_i$ ):
2   | HashTable_Insert( $p_i$ );
3 Function HashTable_Insert( $p_i$ ):
4   | for  $j = 1$  to  $d$  do
5   |   | if  $\mathcal{D}_j[g_j(f_i)]$  is empty or  $\mathcal{D}_j[g_j(f_i)].ID = f_i$  then
6   |   |   | Update_Bucket( $\mathcal{D}_j[g_j(f_i)], p_i$ );
7   |   |   | return;
8   |   | else if  $j = d$  then
9   |   |   | Send_And_Clear_Bucket( $\mathcal{D}_j[g_j(f_i)]$ );
10  |   |   | Update_Bucket( $\mathcal{D}_j[g_j(f_i)], p_i$ );
```

Algorithm 3: Insertion workflow of CellSketch

Input: A packet p_i of flow f_i

```

1 Function Update_Bucket( $p_i$ ):
2   |  $t_i \leftarrow$  queuing delay of  $p_i$ ;
3   | //  $\mathcal{C}_0[t_i]$  denotes the counter whose interval contains  $t_i$ 
4   | if  $\mathcal{C}_0[t_i]$  does not overflow then
5   |   |  $\mathcal{C}_0[t_i] \leftarrow 1$ ;
6   | else
7   |   | for  $j = 1$  to  $v$  do
8   |   |   | CellSketch_Insert( $f_i, t_i, j, \&\mathcal{C}_0[t_i], 1$ );
9 Function CellSketch_Insert( $f_i, t_i, j, address, l$ ):
10 |  $h = \text{hash}_{j,l}(f_i \oplus address)$ ;
11 | if  $\mathcal{C}_l[h][t_i]$  does not overflow then
12 |   |  $\mathcal{C}_l[h][t_i] \leftarrow 1$ ;
13 | else if  $l < L$  then
14 |   | CellSketch_Insert( $f_i, t_i, j, \&\mathcal{C}_l[h][t_i], l + 1$ );
```

Algorithm 4: Query workflow of CellSketch

Input: A flow ID f_i **Output:** Delay distribution of f_i

```
1 Function Query_Bucket( $f_i$ ):  
2    $res \leftarrow \{\}$ ;  
3   for  $k = 1$  to  $r$  do  
4     //  $C_0[k]$  denotes the  $k_{th}$  counter in  $C_0$   
5     if  $C_0[k]$  does not overflow then  
6        $res[k] \leftarrow C_0[k]$ ;  
7     else  
8        $res[k] \leftarrow \{\}$ ;  
9       for  $j = 1$  to  $v$  do  
10         $res[k] = \min(res[k], \text{CellSketch\_Query}(f_i, j, \&C_0[k], 1))$ ;  
11   return  $res$ ;  
12 Function CellSketch_Query( $f_i, j, address, l$ ):  
13    $res \leftarrow \{\}$ ;  
14    $h = \text{hash}_{j,l}(f_i \oplus address)$ ;  
15   for  $k = 1$  to  $r$  do  
16     if  $C_l[h][k]$  does not overflow then  
17        $res[k] \leftarrow C_l[h][k]$ ;  
18     else  
19        $res[k] \leftarrow \text{CellSketch\_Query}(f_i, j, \&C_l[h][k], l+1)$ ;  
20   return  $res$ ;
```

References

- [1] Yinda Zhang, Zaoxing Liu, Ruixin Wang, Tong Yang, Jizhou Li, Ruijie Miao, Peng Liu, Ruwen Zhang, and Junchen Jiang. Cocosketch: high-performance sketch-based measurement over arbitrary partial key query. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 207–222, 2021.
- [2] Tong Yang, Jie Jiang, Peng Liu, Qun Huang, Junzhi Gong, Yang Zhou, Rui Miao, Xiaoming Li, and Steve Uhlig. Elastic sketch: Adaptive and fast network-wide measurements. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, pages 561–575, 2018.
- [3] Jiarui Guo, Yisen Hong, Yuhan Wu, Yunfei Liu, Tong Yang, and Bin Cui. Sketchpolymer: Estimate per-item tail quantile using one sketch. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 590–601, 2023.